Computer Olympic POSN KU Center (CAMP2 2568/2569)

Complete Search

Nopadon Juneam
Department of Computer Science
Kasetsart University

The materials mostly follow chapter 3.2 of CPBook 3ed

Outlines

- Basics of Complete Search
- Techniques for Complete Search

Grader Problems

- [ExSearch] Simple Equations
- [ExSearch] Make Primes
- [ExSearch] Bars
- [ExSearch] Midnight Museums

Practice Problems (Vjudge)

- UVa 725 - Division
- UVa 441 - Lotto
- UVa 11565 - Simple Equations ([ExSearch] Simple Equations)
- UVa 11742 - Social Constraints
- UVa 12455 - Bars

Problem Solving Paradigms

- Complete Search (a.k.a. Brute Force, Exhaustive Search)
- Divide and Conquer
- Greedy Approach
- Dynamic Programming

Complete Search

- The **Compete Search** is also known as *Brute Force* or *Exhaustive Search*

- **Compete Search** is a method for solving a problem by traversing the entire (or part of) the search space to obtain the required solution. During the search, we are allowed to prune (choose not to explore) parts of the search space if these parts have no possibility of containing the required solution

When to Use Complete Search?

- Complete Search solutions are usually easy to come up with, but they may have high orders of complexities
  - E.g. the task of enumerating all permutations of {1,2, …, n} requires O(n!) operations
- We should develop a Complete Search solution *when there is no other algorithm available*, or *when better algorithms exist, but are overkilled as the input size happens to be small*
- *** Many programming problems do have solutions that are better than Complete Search*

Some Example (1)

- Given an array A containing n small integers, where n <10K, and all the integers in A are upper bounded by 100K. Consider what would happen if we attempt each of the following problems with Complete Search?

  - **Problem 1:** Find the largest and the smallest element of A
  - **Problem 2:** Find the *i*-th smallest element in A
  - **Problem 3:** Find the largest gap g such that x,y in A and g = |x-y|
  - **Problem 4:** Find the longest increasing subsequence of A

Some Example (2)

A	10	7	3	5	8	2	9

- **Problem 1:** Find the largest and the smallest element of A
- **Problem 2:** Find the *i*-th smallest element in A
- **Problem 3:** Find the largest gap g such that x,y in A and g = |x-y|
- **Problem 4:** Find the longest increasing subsequence of A

Problem 1

- **Problem 1:** Find the largest/smallest element in the array A
- For problem 1, we can try each element of A and check if it is the current largest (or smallest) element seen so far. *This implies an O(n) Complete Search solution*

Problem 2

- For problem 2, we can use the solution above to find the smallest value and replace it with a large value (e.g. 1M) to delete it. Then, we can proceed with finding the smallest value again (i.e., the second smallest value in the original array)
  - By repeating the process *i* times, we will find the *i*-th smallest value. *This Complete Search solution runs in O(in) time.* However, if *i* = n/2, the solution will run in $O(n^2)$
  - A better solution would be first sorting the array A in O(n log n) time, and then return the value of A[i-1] as answer.
  - Note that there is an even better solution that runs in O(n) expected time (the randomized QuickSelect algorithm)

Problem 3

- For problem 3, we can consider all possible two integers x, y in A, checking if the gap between them is the largest. This Complete Search solution runs in $O(n^2)$ time
  - In fact, it can be proved that the gap g can be obtained by finding the difference between the smallest and largest elements of A. Moreover, these two integers can be found in O(n) using the Complete Search solution for problem A

Problem 4

- For problem 4, we can try all possible subsequences to find the lonest increasing one. However, this solution, which runs in $O(2^n)$, is not feasible at all as the value of n is upper bounded by 10K
  - However, there are a simple $O(n^2)$ Dynamic Programming solution, and the faster O(n log n) Greedy solution for problem 4

Iterative Complete Search

- Techniques to be mastered
  - Nested loops
  - Pruning
  - Enumerating permutations
  - Enumerating subsets

UVa 725 – Division (1)

- <u>Problem statement</u>: Given an integer N, $2 \le N \le 79$, find all pairs of 5-digit numbers, says, **abcde** and **fghij** such that **abcde**/**fghij** = N, and **abcdefghij** must be all distinct

UVa 725 – Division (2)

- <u>Problem statement</u>: Given an integer N, $2 \le N \le 79$, find all pairs of 5-digit numbers, says, **abcde** and **fghij** such that **abcde**/**fghij** = N, and **abcdefghij** must be all distinct
- Iterative Complete Search solution I:
  - Enumerate all possible pairs for **abcde** and **fghij,** and check if they give the right answer.
  - This implies the search space of size roughly 50K*50K = 250M

UVa 725 – Division (3)

- Given an integer N, 2 ≤ N ≤ 79, find all pairs of 5-digit numbers, says, **abcde** and **fghij** such that **abcde/fghij** = N, and all the digits **abcdefghij** must be different

- Iterative Complete Search solution II:
  - Try all possible numbers for **fghij**.Then, compute **abcde** = N*__fghij__, and check if **abcdefghij** are all different.
  - The size of the search space now reduces to 100K/N, and is upper bounded by 50K for N = 2
  - This solution requires about at most 50K*10 = 500K operations, and can be implemented by *two nested loops*

UVa 441 – Lotto (1)

- <u>Problem statement</u>: Given $6 < k < 13$ integers, enumerate all possible subsets of size 6 of these integers in sorted order

UVa 441 – Lotto (2)

- <u>Problem statement</u>: Given $6 < k < 13$ integers, enumerate all possible subsets of size 6 of these integers in sorted order
- Iterative Complete Search solution:
  - As the size of subsets is fixed to be 6, the easiest solution would be to enumerate all possible such subsets. This can be done with six nested loops
  - The size of the entire search space is exactly $\binom{k}{6}$.
    For k = 12, the size is $\binom{k}{6} \approx$ 1K

Combination (Binomial Coefficient)

- How many different ways one can choose k items from a set of n items without repetition and order?

$$\binom{n}{k} = \frac{n\cdot(n-1)\cdot(n-2)\cdots(n-k+1)}{k\cdot(k-1)\cdot(k-2)\cdots 1} = \frac{n!}{k!(n-k)!}$$

- **<u>Quick fact:</u>** $\binom{n}{k} = O(n^k)$

UVa 11565 – Simple Equations (1)

- <u>Problem statement</u>: Given three integers A, B, and C ($1 \le A, B, C \le 10K$), find three other distinct integers x, y, and z such that $x + y + z = A$, $xyz = B$, and $x^2 + y^2 + z^2 = C$

UVa 11565 – Simple Equations (2)

- <u>Problem statement</u>: Given three integers A, B, and C ($1 \le A, B, C \le 10K$), find three other distinct integers x, y, and z such that $x + y + z = A$, $xyz = B$, and $x^2 + y^2 + z^2 = C$

- Iterative Complete Search solution I:

  - First, we need to address the possible range of values for x, y, z. The equation $x^2 + y^2 + z^2 = C$ is a good starting point

  - Assume C = 10K. Then, x in [-100, 100] as $y^2 + z^2 \ge 0$. Similarly, y and z in [-100, 100]. This forms a search space of size $\approx 201^3 = 8M$

UVa 11565 – Simple Equations (3)

- <u>Problem statement</u>: Given three integers A, B, and C ($1 \le A, B, C \le 10K$), find three other distinct integers x, y, and z such that $x + y + z = A$, $x \cdot y \cdot z = B$, and $x^2 + y^2 + z^2 = C$

- Iterative Complete Search solution II:
  - Assume C = 10K. Then, we know x in [-100, 100] as $y^2 + z^2 \ge 0$. Similarly, we know y and z in [-100, 100]. This forms a search space of size $\approx 201^3 = 8M$
  - Further, assume B = 10K and x has the smallest value. We can use the equation $x \cdot y \cdot z = B$ to derive $x < \sqrt[3]{B} \approx 22$. We now reduce the ranges for x, y, z to be [-22, 22]. Thus, the size of the search space reduces to 45 x 201x 201 < 2M

UVa 11472 – Social Constraints (1)

- <u>Problem statement</u>: There are $0 < n \leq 8$ movie goers. They will sit in the front row in n consecutive open seats. There are $0 \leq m \leq 20$ seating constraints among them, i.e., movie goer a and movie goer b must be at most (or at least) c seats apart

UVa 11472 – Social Constraints (2)

- <u>Problem statement</u>: There are $0 < n \leq 8$ movie goers. They will sit in the front row in n consecutive open seats. There are $0 \leq m \leq 20$ seating constraints among them, i.e., movie goer a and movie goer b must be at most (or at least) c seats apart

- Iterative Complete Search solution:

  - Enumerate all possible permutations for seating arrangements and count all the permutations that pass all the constraints. This implies the search space of size $m \cdot n!$
    (upper bounded by $20 \cdot 8! \approx 800K$)

Enumerate Permutations using STL <algorithm>

```
int n = 5;
vector<int> perm(n);
for (int i = 0; i < n; i++) perm[i] = i + 1;

do {
    for (int i = 0; i < n; i++) {
        printf("%d ", perm[i]);
    }
    printf("\n");

} while (next_permutation(perm.begin(), perm.end()));
```

UVa 12455 – Bars (1)

- <u>Problem statement</u>: Given a set L containing $1 < n \leq 20$ integers, is there a subset of L that sums to another given integer X

UVa 12455 – Bars (2)

- <u>Problem statement</u>: Given a set L containing $1 < n \leq 20$ integers, is there a subset of L that sums to another given integer X?
- Iterative Complete Search solution:
  - Try 2^n all possible subsets of L. Sum the selected integers in each subset in O(n) time. Check if the sum equals to X.
  - The overall time complexity of this solution is $O(n \cdot 2^n)$. If we check with the largest test case, i.e., when n = 20, then the solution requires about $\approx 20*2^{20}$ = 21M operations

Enumerate Subsets using Bit Manipulation

- Use the binary representation of integer from 0 to 2^n-1 to describe all possible subsets. This bit manipulation operations are very fast

```
for(int i=0; i < (1<<n); i++) {  // for each possible subset, O(2^n)
    int sum = 0;
    for(int j=0; j<n; j++) {        // check membership, O(n)
        if(i & (1 << j)) {          // check if bit j is turned on in subset i
            sum += l[j];
        }
        if (sum == X)
            break;
}
```

Rule of Thumb

- There are n! permutations of n objects. We can only go through all permutations when $n \leq 11$

- There are 2^n possible subsets of n objects. We can only go through all subsets when $n \leq 25$

- **<u>Quick fact:</u>** $2^n \leq n!$, for $n \geq 4$.

Backtracking (1)

- We have seen two ways (enumerating permutations and possible subsets) to go through a complex search space, but both of the solutions were rather specific
- Thus, it would be nice to have a more general “framework”
- The framework is called “*recursive backtracking*” or just “*backtracking*”

Backtracking (2)

- The structure of a typical backtracking solution
  - Defining *states* and *transitions*
    - Always have one *initial state*
    - Some states are *partial*; some are complete
    - Transitions cause a state to expand to possible next states. The set of all possible states forms the search space

Backtracking (3)

- The basic idea:
  - Start with the initial state
  - Use recursion to traverse all states by going through the transitions
  - If the current state is invalid, then terminating the branch (*pruning*)
  - While traversing, check if it is a complete state and terminate.

Backtracking (4)

- General solution form:

```
state S;

void generate() {
    if (!is_valid(S))
        return;

    if (is_complete(S))
        print(S);

    foreach (possible next move P) {
        apply move P;
        generate();
        undo move P;
    }
}

S = empty state;
generate();
```

Enumerate All Subsets with Backtracking

```
const int n = 5;
bool pick[n];

void generate(int at) {
    if (at == n) {
        for (int i = 0; i < n; i++) {
            if (pick[i]) {
                printf("%d ", i+1);
            }
        }
        printf("\n");
    } else {

        // either pick element no. at
        pick[at] = true;
        generate(at + 1);

        // or don't pick element no. at
        pick[at] = false;
        generate(at + 1);
    }
}

generate(0);
```

Enumerate Permutations with Backtracking (1)

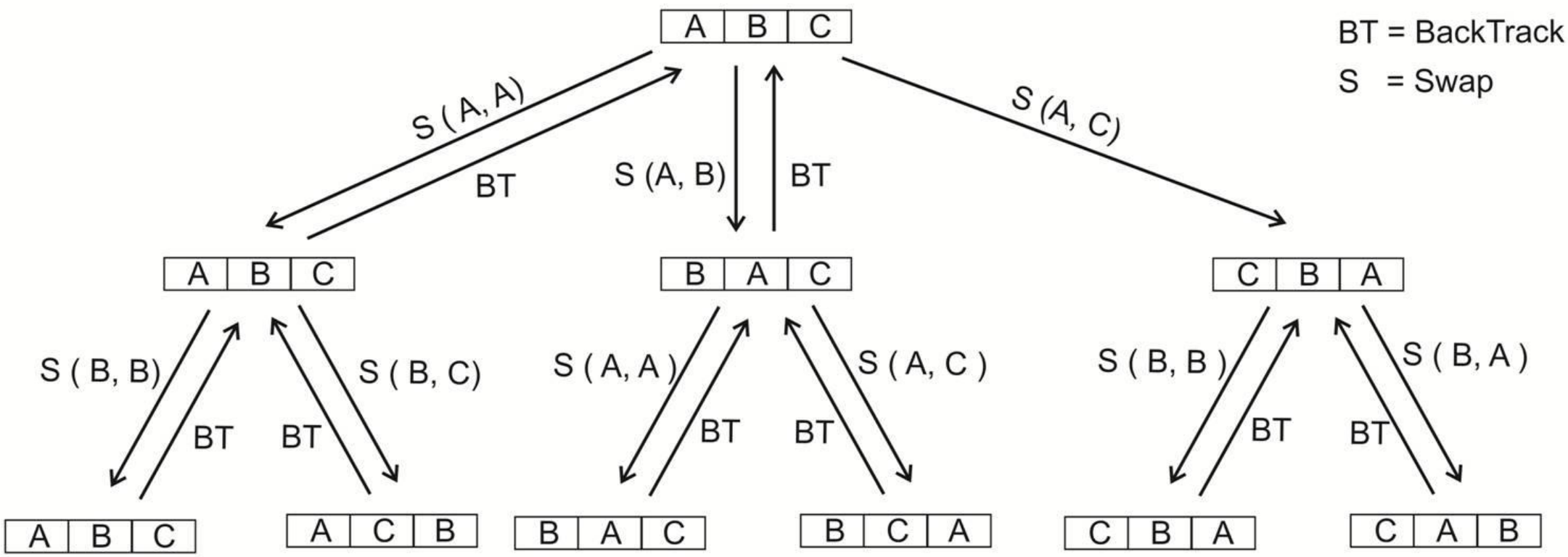
- **<u>Simple recursive algorithm:</u>** The algorithm generates all permutation of a collection n objects as follows:

  - If n=1, print the collection (the current permutation)

  - Otherwise, for each 1 ≤ i ≤ n, swap the element on position 1 with the element on position i. After each swap, the algorithm immediately recurses on the last n-1 elements. Once the recursive call terminates, the algorithm backtracks (restore its state as before the recursion) by swapping the element on position 1 back to the original position i

- **This simple recursive algorithm has time complexity O(n*n!), and is still workable for most problems

Enumerate Permutations with Backtracking (2)



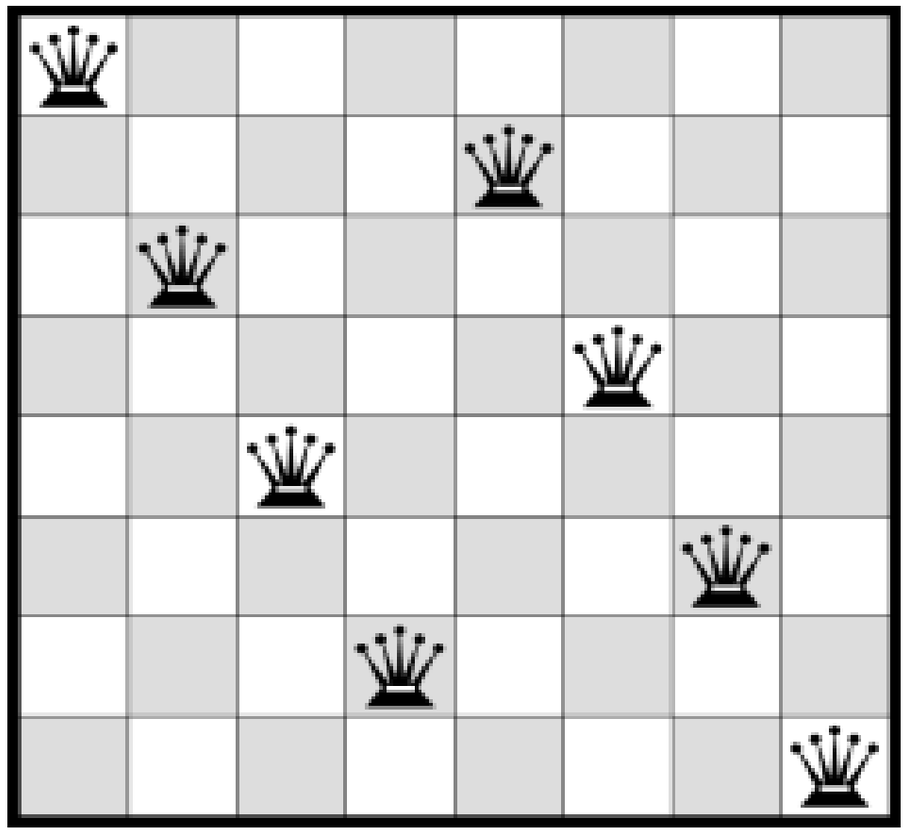
https://tutorialcup.com/interview/string/program-print-permutations-string.htm

Enumerate Permutations with Backtracking (3)

```
void permute(string& A, int l, int r)
{

    if (l == r)
        cout << A << endl;
    else {
        for (int i = l; i <= r; i++) {
            swap(A[l], A[i]);

            permute(A, l + 1, r);

            swap(A[l], A[i]);
        }
    }
}
```

UVa 750 – 8 Queens (1)

- <u>Problem statement</u>: Place 8 queens on an 8x8 board such that no two queens attack each other, given the position of one of the queen



UVa 750 – 8 Queens (2)

- Most naïve solution:
  - Iteratively enumerate all combinations of 8 cells out of 8x8 = 64 cells. Then, see if the 8 queens can be placed at these position
  - The search space due to this solution is $\binom{64}{8} \approx$ 4B
- Better, but still naïve solution:
  - We know that *each queen can only occupy one column*. So, we can place exactly one queen in each column
  - The search space now goes down to $8^8 \approx$ 17M

UVa 750 – 8 Queens (3)

- Fast and better solution:
  - Observe that *no two queens can share the same column or the same row*
  - With the above observation, we can model the original problem as the problem of finding valid permutations of 8! row positions
  - The search space due to this solution is 8! ≈ 40K. This solution is fast enough!!

UVa 750 – 8 Queens (4)

- Recursive backtracking solution (even better solution):
  - Since we also know that no two queens can share any of the two diagonal lines. Let queen A be at (i, j) and queen B be at (k,l). These two queens attack each other if and only if abs(i-k) == abs (j-l)
  - Our solution places the queens one by one in column 0 to 7, observing all the related constraints. This solution implies the search space of size smaller than 8!

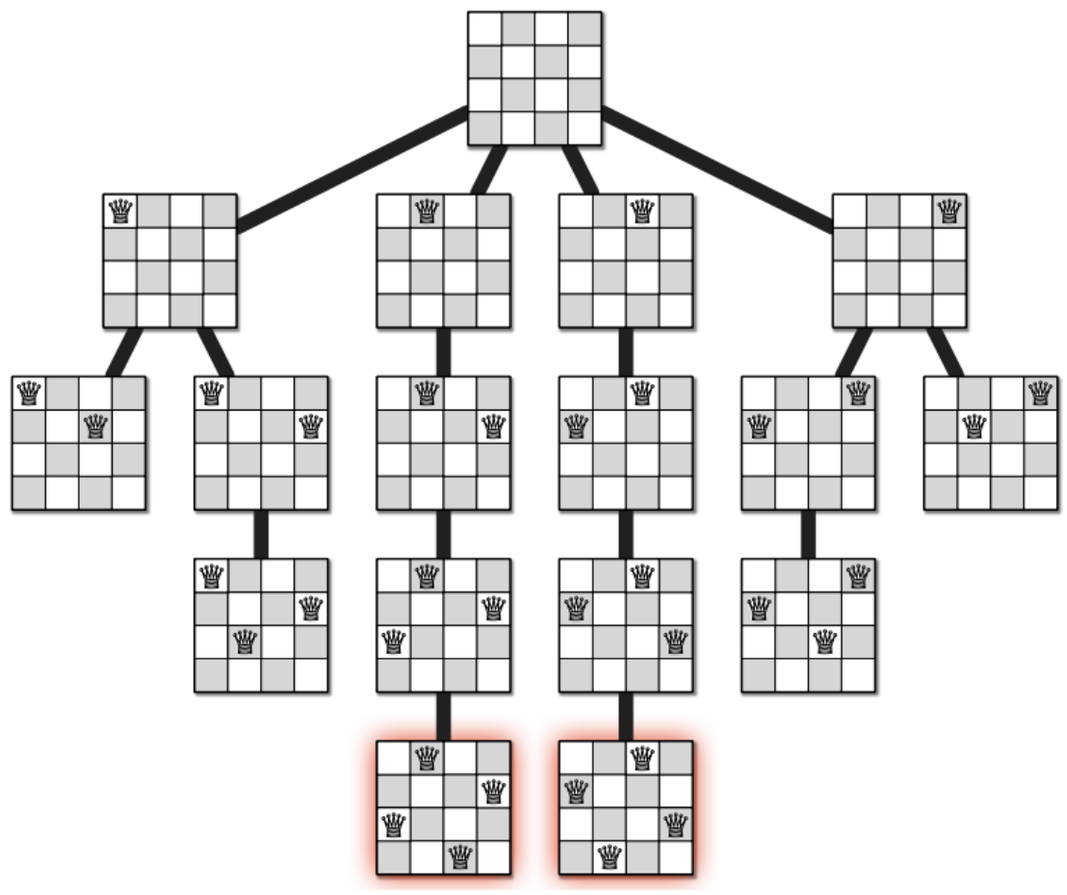
UVa 750 – 8 Queens (5)

- Recursive backtracking solution:

```
int row[8], TC, a, b, lineCounter;            // ok to use global variables

bool place(int r, int c) {
  for (int prev = 0; prev < c; prev++)     // check previously placed queens
    if (row[prev] == r || (abs(row[prev] - r) == abs(prev - c)))
      return false;         // share same row or same diagonal -> infeasible
  return true;
}

void backtrack(int c) {
  if (c == 8 && row[b] == a) {          // candidate sol, (a, b) has 1 queen
    printf("%2d %d", ++lineCounter, row[0] + 1);
    for (int j = 1; j < 8; j++)
         printf(" %d", row[j] + 1);
    printf("\n");
  }
  for (int r = 0; r < 8; r++)                          // try all possible row
    if (place(r, c)) {          // if can place a queen at this col and row
      row[c] = r; backtrack(c + 1);      // put this queen here and recurse
}   }
```

UVa 750 – 8 Queens (6)

- Recursive backtracking solution:



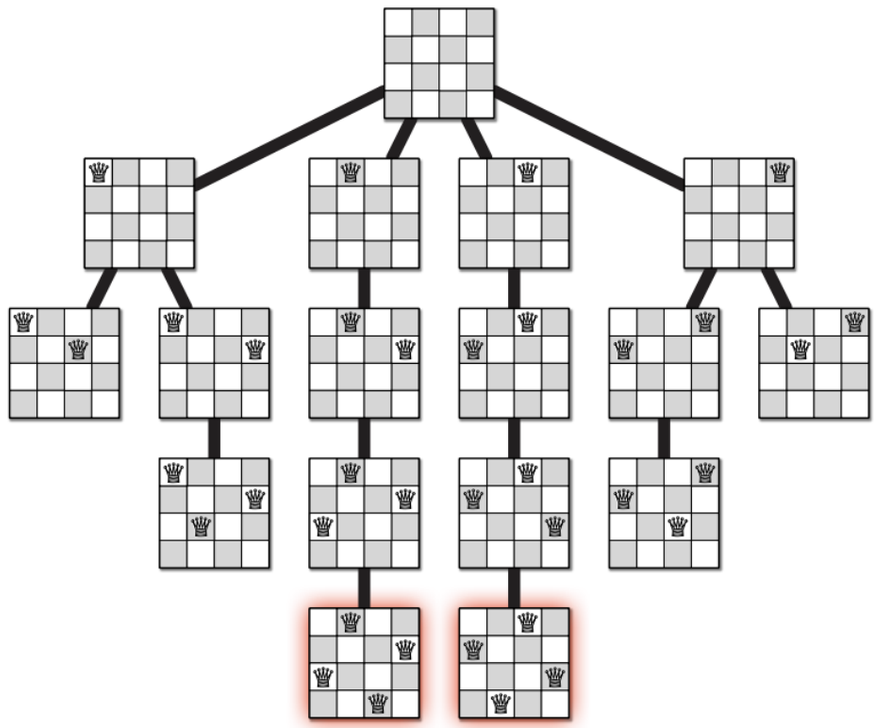
https://codeahoy.com/learn/recursion/ch10/

UVa 750 – 8 Queens (7)

- We have talked about two different kinds of complete search solutions
  - Iterative complete search solution (enumerating all possible permutations with many nested loops)
  - Recursive backtracking solution
- **Remarks:** The second solution can be generalized to the problem of placing n queens (still, the sub O(n)! search space). However, the first solution will suffer because we do not know how many nested loops to perform

Backtracking as Tree Search

- Often, the search space due to recursive backtracking can be modeled as a tree with nodes representing states, and links representing possible changes between states



- A ***recursive backtracking*** can be thought as a process of searching the tree (i.e., more like tree traversal) for a particular leaf or node (goal state) starting from root (initial state)